

Centre No: _____

Centre Name: _____

Code: _____ Subject: _____

Candidate Number: _____ Candidate Name: Henry Dong

Signature: _____

Do not
write in
this
margin

- (a) 24
30
- (b) 1-2-3-4-90
- (c) odd
even
- (d) returns void simply means return nothing but it can change value of variables, like countUp(int n) in (b)
returns value means that the method can be treated as a value as it returns one. It can be used in print, or to do calculations, like area(b,a) + area(2,3) or
System.out.println(label(7))

- (a) (i) ~~zero~~ 0 to 5
(ii) 1 to 6
(iii) It has a for loop which repeats for 4 times to simulate rolling a dice four times
(iv) more readable and maintainable.

In the question, the simple have rnd makes function more readable and easier to maintain.

~~(a) (i) for (int i=0; i<5; i++) {
System.out.println(rnd.nextInt(6)+1);
}~~

(b) ~~3~~
(i) ~~public static int roll() {
returns (rnd.nextInt(6)+1);
}~~

```

5)) for (int i=0; i<5; i++) {
    System.out.print(roll());
    System.out.print("\n");
}

```

(C)

(i) 0 to 1 exclusive
circle (points inside the circle)

(ii) $A_{\text{circle}} = \pi r^2 = 0.25\pi$

$$A_{\text{square}} = 1$$

$$\frac{0.25\pi}{1} = \frac{\text{cnt}}{N}$$

$$\pi = 4 \times \frac{\text{cnt}}{N}$$

(iii) ~~the~~ the data type of π is double, so we need a double
type ~~not integer~~ ~~double~~ to multiply cnt/N and get final answer,
or there will be an error.

Q3

```

9) FUNCTION nextTerm (n: INTEGER) RETURNS INTEGER
    IF n MOD 2 = 0 THEN
        RETURN n/2
    ELSE RETURN (3*n+1)
    ENDIF
ENDFUNCTION

```

```

b) FUNCTION steps (n: INTEGER) RETURNS INTEGER
    DECLARE num is, cnt: INTEGER
    cnt ← 0
    is
    num ← 0
    WHILE NOT
    INPUT num
    WHILE NOT (num = 1) is

```

Q3b)

```

num ← nextTerm (num)
cnt ← cnt + 1
ENDWHILE
RETURN cnt

```

c)

Because we don't know when the nextTerm() returns exactly, so ~~we~~ we have to use while loop to test if it returns one instead of using counter ~~loop~~ control loop.

d)

```

DECLARE cnt: INTEGER
INPUT n
WHILE NOT (n=1)
n ← nextTerm(n)
n ← n/2
OUTPUT n
n ← nextTerm(n)
ENDWHILE

```

OUTPUT & "Steps: " & steps(n)

6:

6 3 10 5 16 8 4 2 1

Steps: 8

~~7 22 11 30 15 46 23 70 35~~

7:

7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

Steps: 16